

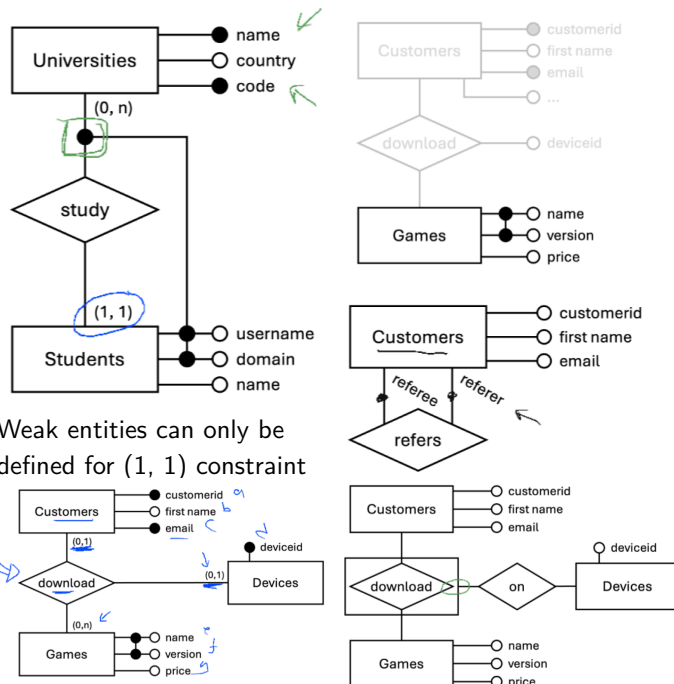
CS4221 Midterm Cheatsheet

by Jonathan Loh, River Koh ([GitHub](#))

L02: Conceptual Modelling (ER)

ER to Relational Mapping Rules

- **Strong Entity:** Becomes a table. Attributes become columns. PK remains.
- **Weak Entity:** Table includes its own attributes + PK of the identifying dominant strong entity. PK is the combination of both.
- **1:N Relationship:** Merge relationship into the entity on the 'N' side (add PK of '1' side as Foreign Key).
- **M:N Relationship:** Create a new table. PK is the combination of PKs from participating entities.
- **1:1 Relationship:** Merge into the entity with mandatory participation to minimize NULLs.
- **Nuance - Minimization of NULLs:** Nulls should only exist if they represent missing data meaning, not as an artifact of bad schema translation. Default constraint in SQL should be NOT NULL unless explicitly justified.
- **Derived Attributes:** Dotted line



Weak entities can only be defined for (1, 1) constraint

L03: Data Warehousing & OLAP

OLTP vs OLAP

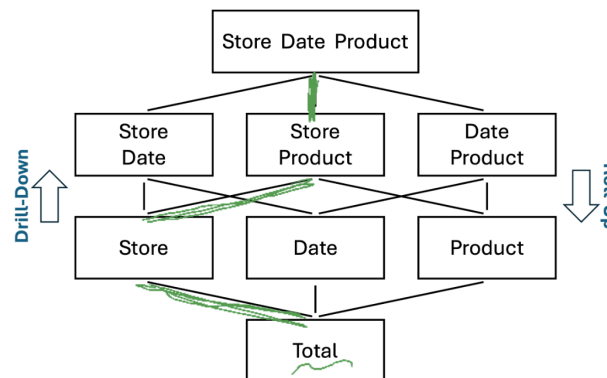
- **OLTP:** Operational, normalized, short updates, current data. Designed to avoid read-write locks blocking each other. (Typical operational database)
- **OLAP:** Analytical, denormalized, bulk reads/aggregations, historical data (Data warehouses)

Dimensional Modelling

- **Fact Tables:** Contains business metrics (measures) and foreign keys to dimension tables. Highly massive.
- **Dimension Tables:** Descriptive attributes. Uses surrogate keys (synthetic) rather than natural keys.
- **Star Schema:** Dimension tables are fully denormalized. Fast queries, simpler joins.
- **Snowflake Schema:** Dimension tables are normalized. Saves space, complex joins.

OLAP Operations & SQL Nuances

- **Slice/Dice:** Filtering subsets of a data cube using WHERE.
- **SQL CUBE(A, B):** Generates 2^n grouping sets: (A, B), (A), (B), ().
 - ▶ SELECT product, region, SUM(qty) FROM sales GROUP BY CUBE (product, region)
- **SQL ROLLUP(A, B):** Generates hierarchical sets: (A, B), (A), ().
 - ▶ e.g. SELECT product, region, SUM(qty) FROM sales GROUP BY ROLLUP (product, region)
- In ROLLUP/CUBE output, aggregated columns for subtotals will appear as **NULL**.
- **Drill-Down:** Implemented using GROUP BY



L04: Relational Algebra

Basic Operations

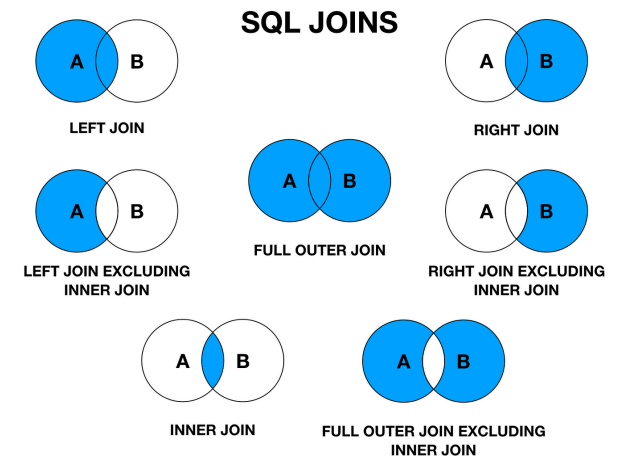
- **Selection (σ_c):** Filters rows based on condition c .
- **Projection (π_L):** Extracts specific columns L . Removes duplicates automatically in relational algebra (pure sets).
- **Cross Product ($\times$):** Cartesian product. $R \times S$ has $n + m$ columns and $n \times m$ rows.
- **Rename Attr:** $\rho_{b \leftarrow a}$
- **Rename Relation:** $\rho(R, S)$

Set Operations (Requires union-compatibility)

- **UNION ($\cup$):** Combines tuples, removing duplicates.
- **INTERSECT ($\cap$):** Tuples present in both R and S .
- **Minus / Difference EXCEPT ($\setminus$):** $R \setminus S$ returns tuples in R but not in S .

Join Operations

- **Theta Join ($\bowtie_\theta$):** $R \bowtie_\theta S \equiv \sigma_\theta(R \times S)$.
- **Natural Join ($\bowtie$):** Equi-join on all matching column names, keeping only one copy of matching columns.
- **Left Outer Join ($\ltimes$):** Keeps all rows from the left table. Unmatched right columns become NULL.
- **Right Outer Join ($\rtimes$):** Keeps all rows from right table.
- **Full Outer Join ($\ltimes$):** Keeps all rows from both tables, filling NULLs where there are no matches.
- **Semi-Join ($\ltimes$):** Returns rows in R that have a match in S . (Optimized for EXISTS).
- **Anti-Join ($\nexists$):** Returns rows in R that DO NOT have a match in S . (Optimized for NOT EXISTS).



Equivalence & Optimization

- **Selection Pushdown:** $\sigma_c(R \bowtie S) \equiv \sigma_c(R) \bowtie S$ (if condition c only applies to R). Reduces rows early!
- **Projection Distribution:** $\pi_L(E_1 \cup E_2) \equiv \pi_L(E_1) \cup \pi_L(E_2)$.

L05: SQL Tuning I (Indexes)

EXPLAIN & Query Plans

- EXPLAIN: Est. startup..total cost, rows, width in Bytes

Query Plan
Aggregate (cost=849.87..849.88 rows=1 width=8)
-> Nested Loop (cost=11.33..849.65 rows=89 width=2)
-> Hash Join (cost=11.05..822.10 rows=93 width=6)
Hash Cond: (s.i_id = i.i_id)
-> Seq Scan on stocks s (cost=0.00..692.12 rows=44912 width=10)
-> Hash (cost=11.04..11.04 rows=1 width=4)
-> Seq Scan on items i (cost=0.00..11.04 rows=1 width=4)
Filter: ((i.name)::text = 'Aspirin')::text
-> Index Scan using warehouses_pkey on warehouses w (cost=0.28..0.30 rows=1 width=4)
Index Cond: (w.id = s.w_id)
Filter: ((w.country)::text = 'Indonesia')::text

- EXPLAIN (ANALYZE): Actual execution times, rows, loops, filtered rows for each filter

Query Plan
Aggregate (cost=849.87..849.88 rows=1 width=8) (actual time=4.122..4.125 rows=1 loops=1)
-> Nested Loop (cost=11.33..849.65 rows=89 width=2) (actual time=0.138..4.113 rows=72 loops=1)
-> Hash Join (cost=11.05..822.10 rows=93 width=6) (actual time=0.079..3.978 rows=76 loops=1)
:
-> Hash (cost=11.04..11.04 rows=1 width=4) (actual time=0.033..0.034 rows=1 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 9kB
-> Seq Scan on items i (cost=0.00..11.04 rows=1 width=4)
(actual time=0.023..0.031 rows=1 loops=1)
:
:
:
Planning Time: 2.326 ms
Execution Time: 4.156 ms

- **Temporary Table** : CREATE TEMPORARY TABLE stocks_agg AS SELECT * FROM stocks WHERE ...
 - No indexes, no implicit statistics (need ANALYZE temp_t)

Catalogue and Statistics

- **pg_tables**: name, index, triggers in DB
- **pg_attribute**: table columns, one row for every column
- **pg_statistics**: created by ANALYZE, approximate statistics about contents in DB
- **pg_stats**: allows access to rows of pg_statistics that user has permissions for to read

Scan Types

- **Sequential Scan**: Reads every page.
- **Index Scan**: Traverses B-Tree, then fetches rows from the heap. Causes random I/O.

- **Index-Only Scan**: Finds **all** requested columns directly in the index. No heap fetch!
- **Covering Index**: CREATE INDEX idx ON T(A) INCLUDE (B)
- **Predicate Index**: CREATE INDEX idx ON T(A) WHERE A > 600

L06: SQL Tuning II & Maintenance

Join Implementations

- **Nested Loop Join**: Outer table scanned once, inner table scanned for every outer row. Good if inner table is indexed.
- **Hash Join**: Builds a hash table on the inner relation. Favored for large unsorted tables. Requires work_mem.
- **Merge Join**: Sorts both inputs, then merges. Best if inputs are already sorted by B-Tree indexes.

Table Maintenance

- **VACUUM**: Reclaims space from dead tuples (MVCC artifacts).
- **VACUUM FULL**: Rewrites table to disk, shrinking file. Takes an exclusive lock.
- **CLUSTER**: Rewrites table so physical order matches an index.

Common SQL Nuances & Midterm Specifics

DROP vs DELETE vs TRUNCATE

- **DELETE (DML)**: Removes rows row-by-row. Logs each deletion. Slower, but triggers fire. Can use WHERE.
- **TRUNCATE (DDL-like DML)**: Deletes all rows instantly by deallocating pages. No triggers fired. Resets IDENTITY/SERIAL columns. Cannot use WHERE.
- **DROP TABLE (DDL)**: Completely removes the table data, structure, indexes, and privileges.

EXCEPT vs NOT IN vs NOT EXISTS

- EXCEPT: Set operation, automatically removes duplicates.
- NOT IN: **Danger**: Fails completely (returns 0 rows) if the subquery returns ANY NULL values.
- NOT EXISTS: Safer, usually optimized into an Anti-Join.

Other Traps

- **NULL Comparisons**: x = NULL evaluates to UNKNOWN (treated as false in WHERE). Always use x IS NULL.

- **WHERE vs HAVING**: WHERE filters rows **before** aggregation. HAVING filters groups **after** aggregation.
- **UNION vs UNION ALL**: UNION sorts and removes duplicates (expensive). Use UNION ALL if duplicates are acceptable or impossible.
- **Execution Order**: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. (You cannot use an alias defined in SELECT within the WHERE clause!).

Cross-only Join Cheatsheet (RA)

Core Trick

- **Theta Join**: $R \bowtie_{\theta} S \equiv \sigma_{\theta}(R \times S)$
- **Equijoin**: $R \bowtie_{R.a=S.b} S \equiv \sigma_{R.a=S.b}(R \times S)$

Natural Join (only using $\times, \sigma, \pi, \rho$) Let $R(A, B)$ and $S(B, C)$.

- Rename to avoid duplicate attribute names: $S' := \rho_{B' \leftarrow B}(S)$
- Cross + select on equality: $T := \sigma_{R.B=S'.B'}(R \times S')$
- Drop duplicate join column: $R \bowtie S \equiv \pi_{A,B,C}(T)$

Self-Join (needs rename)

- $\sigma_{\theta}(\rho_{R1}(R) \times \rho_{R2}(R))$

Semi-Join (EXISTS / ANY vs IN) from cross join

- Definition idea: join then keep only left attrs
- $R \ltimes_{\theta} S \equiv \delta(\pi_{(R)}(\sigma_{\theta}(R \times S)))$

Anti-Join (NOT EXISTS / LEFT JOIN) from cross join

- $R \triangleright_{\theta} S \equiv R \setminus (R \ltimes_{\theta} S)$

Assignment 1 identities

- $R \bowtie_{[\text{cond}]} S \equiv \delta(\sigma_{[\text{cond}]}(R \times S)) \cup (R \triangleright_{[\text{cond}]} S)$
- $R \div S \equiv \pi_{[R-S]}(R) \setminus (\pi_{[R-S]}(\pi_{[R]}(\pi_{[R-S]}(R) \times S) \setminus R))$